Customer Billing System

Alex Montes de Oca 2071316

Computer Engineering Lab

Customer and Product Structures

- Singly Linked Lists
- Product name to be dynamically allocated as these are harder to predict
- Customer fields have a defined average/max length that can be safely predicted

```c
typedef struct Product Product;
struct Product{
    char *name;
    float price;
    Product *prodNext;
};
```

```c
/* Maximum capacity for arrays */
#define NAME_MAX 70
#define ADDR_MAX 100
#define EMAIL_MAX 60
```

```c
typedef struct Customer Customer;
struct Customer{
    char name[NAME_MAX+1];
    char address[ADDR_MAX+1];
    char email[EMAIL_MAX+1];
    Product *cart;
    Customer *cusNext;
};
```

Checking File Integrity

```
#include <openssl/evp.h>
#include <openssl/sha.h>
```

- OpenSSL
- Hash creation, comparison, and updating
- Creation of external .txt file to verify hash codes

```
int generateFileHash(
int compareHashes(cons
int writeHashToFile(c
int readHashFromFile(
```

```
// Check integrity of files
unsigned char generatedHash[SHA256_DIGEST_LENGTH];
unsigned char storedHash[SHA256_DIGEST_LENGTH];

// Verify catalogue csv file
if (readHashFromFile("barcelona_catalogue_hash.txt", storedHash, SHA256_DIGEST_LENGTH) == 0 &&
    generateFileHash("barcelona_catalogue.csv", generatedHash) == 0 &&
    !compareHashes(generatedHash, storedHash, SHA256_DIGEST_LENGTH))
{
    fprintf(stderr, "File integrity verification failed for barcelona_catalogue.csv.\n");
    return EXIT_FAILURE;
}
```

Integrity checked upon every successive execution.

```
// calculate hash for file integrity check
unsigned char hash[SHA256_DIGEST_LENGTH];

if (generateFileHash("barcelona_catalogue.csv", hash) == 0) {
    writeHashToFile("barcelona_catalogue_hash.txt", hash, SHA256_DIGEST_LENGTH);
}

// Generate and store the hash for customers.csv
if (generateFileHash("customers.csv", hash) == 0) {
    writeHashToFile("customers_hash.txt", hash, SHA256_DIGEST_LENGTH);
}
```

Hash calculation on first-time program execution, .txt file creation.

Issues Experienced & Implemented Solutions

scanf() and Input Buffer

In different and often random cases, it became necessary to enter a newline character in order to continue with the program. A function was created, but this kept being a repeated (small) issue.

```
void clearInputBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
```

addToCart affecting Catalogue

On simulating a purchase, products would move directly from the catalogue to the customer's cart, bringing all successive nodes with it. A function was created to copy products and create an independent list.

```
// used for addToCart
Product *createProductCopy(Product *original)
{
```

Editing Product Name

While having to dynamically replace the name of a given product, issues would sometimes occur with memory allocation. Fixed with realloc().

```
char *newName = (char *)realloc(target->name, strlen(name) + 1);
target->name = newName; // change pointer
strcpy(target->name, name);
```

Error Handling in Purchase Simulation

Partially affected by the above, there were cases of infinite loops occurring with menu printings and faulty user input. More error messages and failsafes were added, so no unintended outcome should occur.